

TIMER_IT

TIMER_IT

- 1. Timer Overview
- 2. Core Concepts
 - 2.1 MicroPython Timer Types
 - 2.2 Timer Modes
 - 2.3 ISR Design Principle — Only Count, No Work
 - 2.4 Critical Section Protection — `disable_irq` / `enable_irq`
 - 2.5 `time_ticks_ms()` — Millisecond Timestamp
 - 2.6 Core API
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Imports and Global Variables
 - 5.2 Timer ISR — Minimal Callback
 - 5.3 Starting the Timer
 - 5.4 Main Loop — Cumulative Printing + Critical Section
- 6. Operation Procedure
 - 6.1 Flashing and Running
 - 6.2 Normal Phenomenon
 - 6.3 Verification Methods
- 7. Common Problems

1. Timer Overview

The timer is one of the most commonly used peripherals of an MCU — set a time interval, and the callback function is triggered automatically when the time is up. This experiment uses `machine.Timer` to configure a 10ms periodic timer. The ISR only records the timestamp and the trigger count (it does not toggle the LED or print). The main loop prints the elapsed running time via serial once every 100 triggers (≈1s).

Typical application scenarios:

Application Type	Example
Periodic tasks	LED blinking, timed sensor sampling
Timeout judgment	Communication timeout, long-press button detection
PWM generation	Breathing light, motor speed control
Precise timing	Pulse measurement, frequency counting

2. Core Concepts

2.1 MicroPython Timer Types

The ESP32-S3 has 4 hardware timers (Timer 0~3):

Timer ID	Description
<code>Timer(0)</code>	Hardware timer 0
<code>Timer(1)</code>	Hardware timer 1
<code>Timer(2)</code>	Hardware timer 2
<code>Timer(3)</code>	Hardware timer 3

Do not confuse these with FreeRTOS software timers — `machine.Timer` directly uses the hardware timers.

2.2 Timer Modes

Mode	<code>Timer</code> Constant	Description
One-shot	<code>Timer.ONE_SHOT</code>	Triggers once when the time expires, then stops
Periodic	<code>Timer.PERIODIC</code>	Repeats every <code>period</code> milliseconds

2.3 ISR Design Principle — Only Count, No Work

This experiment demonstrates a core practice: **the ISR only does the lightest operations — recording the timestamp and accumulating the count**. Time-consuming operations such as printing and LED toggling are all placed in the main loop.

ISR does	ISR does not do
<code>time.ticks_ms()</code> reads the timestamp	<code>print()</code>
<code>isr_trigger_count += 1</code> accumulates an integer	LED toggle / GPIO operations
—	<code>time.sleep()</code>

2.4 Critical Section Protection — `disable_irq` / `enable_irq`

When the ISR and the main loop share variables (`isr_capture_time`, `isr_trigger_count`), there is a **race condition risk**: the main loop is interrupted by the ISR halfway through a read, and the variable is modified, causing inconsistent data.

```

irq_state = disable_irq()    # disable IRQs, save the current interrupt state
trigger   = isr_trigger_count # atomic read
now       = isr_capture_time
enable_irq(irq_state)       # restore the interrupt state

```

Reading/writing a 32-bit integer is atomic on the ESP32-S3, but not across two variables — disabling interrupts guarantees that this small section is "indivisible".

2.5 `time.ticks_ms()` — Millisecond Timestamp

```

import time
t = time.ticks_ms()           # returns milliseconds since startup (wraparound-safe)
elapsed = time.ticks_diff(now, start) # safely compute the difference, handles wraparound
                                         automatically

```

`ticks_diff()` has built-in wraparound handling, so it still computes the difference correctly even when `ticks_ms()` wraps back to 0.

2.6 Core API

```

from machine import Timer

tim = Timer(0)                # use timer 0
tim.init(period=10, mode=Timer.PERIODIC, callback=fn) # 10ms period
tim.deinit()                  # stop the timer

```

Parameter	Description
<code>period</code>	Timer period, in milliseconds
<code>mode</code>	<code>Timer.PERIODIC</code> or <code>Timer.ONE_SHOT</code>
<code>callback</code>	Timer callback function, receives one argument <code>tim</code> (the timer object)

3. Hardware Dependencies

This experiment is a pure system-function demonstration and does not involve GPIO peripherals. Serial printing is used to observe the timer timing.

4. Project Architecture and Flow

```

Script execution (single-file .py)
|
|─ isr_capture_time = 0           // timestamp captured by the ISR
|─ isr_trigger_count = 0         // total number of ISR triggers
|─ def timer_callback(tim):      // timer ISR (triggers every 10ms)
|   |─ isr_capture_time = time.ticks_ms() // record the current timestamp

```

```

|     ↳ isr_trigger_count += 1                // increment trigger count
|     ↳ tim = Timer(0)                        // create the timer
|     ↳ tim.init(period=10, PERIODIC, callback) // start: 10ms period
|
|     ↳ while True:                          // main loop
|         ↳ trigger = isr_trigger_count        // read ISR trigger count
|         ↳ if trigger - last_print >= 100:    // print every 100 times (≈1s)
|             ↳ disable_irq()                 // critical section start
|             ↳ trigger = isr_trigger_count; now = isr_capture_time // atomic read
|             ↳ enable_irq(irq_state)          // critical section end
|             ↳ elapsed = ticks_diff(now, start) // compute elapsed time
|             ↳ print("elapsed:", elapsed, "ms") // serial output
|     ↳ last_print_trigger = trigger           // update print threshold

```

5. Source Code Details

Full source code: [5.Source Code](#) → [MicroPython](#) → [1.Basic Peripherals](#) → [4.TIMER_IT](#) → [4.TIMER_IT.py](#)

5.1 Imports and Global Variables

```

from machine import Timer, disable_irq, enable_irq # Timer + IRQ control
import time                                       # ticks_ms / ticks_diff

isr_capture_time = 0 # timestamp captured by the ISR (ms)
isr_trigger_count = 0 # total ISR trigger count

```

`disable_irq` and `enable_irq` are the key new APIs in this experiment — they are used to briefly disable interrupts when reading the ISR-shared variables.

5.2 Timer ISR — Minimal Callback

```

def timer_callback(tim): # timer ISR callback
    global isr_capture_time, isr_trigger_count

    isr_capture_time = time.ticks_ms() # capture trigger time
    isr_trigger_count += 1              # increment count

```

The ISR only uses two lines of variable assignment, taking $< 1\mu\text{s}$. It does not touch GPIO, print, or delay — this is the experiment goal of "don't make the ISR do work".

5.3 Starting the Timer

```

tim = Timer(0) # hardware timer 0
tim.init(period=10, mode=Timer.PERIODIC, callback=timer_callback) # 10ms periodic

print("Timer started (period 10 ms)") # timer started

```



```
Timer started (period 10 ms)
[100] elapsed: 999 ms
[200] elapsed: 1999 ms
[300] elapsed: 2999 ms
[400] elapsed: 3999 ms
[500] elapsed: 4999 ms
[600] elapsed: 5999 ms
[700] elapsed: 6999 ms
[800] elapsed: 7999 ms
```

```
Timer started (period 10 ms)
[100] elapsed: 999 ms
[200] elapsed: 1999 ms
[300] elapsed: 2999 ms
[400] elapsed: 3999 ms
[500] elapsed: 4999 ms
[600] elapsed: 5999 ms
[700] elapsed: 6999 ms
[800] elapsed: 7999 ms
[900] elapsed: 8999 ms
[1000] elapsed: 9999 ms
[1100] elapsed: 10999 ms
[1200] elapsed: 11999 ms
[1300] elapsed: 12999 ms
[1400] elapsed: 13999 ms
```

Due to slight jitter in ISR execution and main-loop scheduling, `elapsed` may not be exactly a whole multiple of 1000ms; ± 1 ms is perfectly normal.

6.3 Verification Methods

Operation	Expected Result
Power up	Shell prints one line per second
Change <code>period=10</code> to <code>period=50</code>	Prints once every 5s (100×50 ms)
Change the threshold <code>100</code> to <code>50</code>	Prints twice per second (every 500ms)
Remove <code>disable_irq</code> / <code>enable_irq</code>	Still works (atomic 32-bit read/write), but not logically rigorous
Add <code>print("ISR")</code> in the ISR	100 prints per second → Shell freezes

7. Common Problems

Symptom	Cause	Solution
Timer does not trigger	<code>period</code> is in milliseconds; wrote <code>10000</code>	Confirm the unit: 10ms = <code>period=10</code>
Shell print interval is unstable	ISR and main loop run in time-sharing; the main loop is preempted by other tasks	Normal phenomenon; use ESP-IDF for high real-time requirements

Symptom	Cause	Solution
No error after removing <code>disable_irq</code>	32-bit integer reads/writes are atomic on the ESP32-S3	But not across two variables — keeping the critical section is correct
<code>elapsed</code> jumps > 1s during printing	Main loop overloaded or GC reclaiming causes the <code>if</code> to be missed	Change the threshold <code>100</code> to <code>50</code> to reduce the single interval
<code>Timer.init()</code> reports an error	The timer is already occupied	<code>tim.deinit()</code> before init, or use another Timer ID